

Machine Learning

Kon Yi

September 18, 2026

Preface

These are my notes for *Machine Learning* (機器學習), taught by Hsuan-Tien Lin (林軒田) at National Taiwan University in the fall of 2026. This preface reproduces the course-introduction handout of 09/09/2026, so that the ground rules sit next to the material they govern; everything after it is the course itself.

Five Reasons for NOT Taking the Course

The course opens, by tradition, with an argument against taking it.

Complicated contents. From a Taiwanese student taking the MIT ML class (translated):

The professor started writing math equations as if he was using some writing accelerator. After class I always felt feeble. The worst part is: I needed to understand the contents as soon as I can. Otherwise I cannot finish the homework and cannot follow up in the next class.

The NTU ML class is designed to be as good as the best classes in the world, so similar things will happen to you. *If you are not willing to be so miserable, ...*

Strict instructor. Will you give me a second chance if I copy homework from other people? No. Could you let me pass because I will be kicked out by the 1/2 rule? No. Will you change my score from F to C? No. How many will pass? Any, if necessary. *If you do not like a strict instructor, ...*

Huge loads. From a student taking ML 2010, posted on BBS (translated):

Only 1 problem requires running 100 test experiments? ($100 \times 10 \text{ min} = 16 \text{ hr}$); I double-checked multiple times — the one who designed this homework is heartless.

This class is two to six times harder than a normal one at NTU: six homework sets are planned, plus the final project, each due within two weeks in general. Homework 0 arrives in the first week — already hard, worth very few points, and there to remind you of the prerequisites. *If you do not want to spend so much time on homework, ...*

Online-learnable. Two of the course's incarnations were invited by NTU as Massive Online Open Courses on NTU-Coursera, *Machine Learning Foundations* and *Machine Learning Techniques*, which cover 70% of what we teach; it is much easier to simply take the recorded version at home. Indeed we will do pseudo-flipped teaching. *If you want to learn at your own pace, ...*

Shallow and old-fashioned. This is not a deep learning class — it surrounds shallow (old) machine learning. Yes, many of the materials are ten or more years old; and yes, AI has knowledge of those materials, just as AI can write code. *If you want to chase newer buzzwords, ...*

From a student in ML 2013, in the final feedback (translated): *“this is a class that makes you experience all kinds of devastation and desperation.”*

May the Brave Ones Stay.

Basic Information

- **Mode:** physical, with a live YouTube screencast and Slido questions (#HTML2026FALL), if everything goes smoothly.
- **Instructor:** Hsuan-Tien Lin, htlin@csie.ntu.edu.tw. Office hour after class, or by appointment.
- **Public course infopage:** <https://www.csie.ntu.edu.tw/~htlin/course/ml26fall/>.
- **Semi-private course platform:** NTU COOL, for announcements, recorded videos and the like — be sure to check emails from NTU COOL frequently.
- **Class chats:** Discord (invitation URL on NTU COOL).
- **Homework grading** (registered students only): Gradescope, linked from NTU COOL.
- **Languages:** teaching in Mandarin; Slido questions in English, traditional or simplified Chinese; homework written in English or Chinese; TA and instructor hours in Mandarin, with English welcome. Don't be afraid of English.

The Course and Its History

- 2008–2012: 3 credits, taught in English.
- 2013–2014: 3 credits, Mandarin, with MOOC.
- 2015: 4 credits, Mandarin, with MOOC.
- 2016–2020: mostly 2 + 2 credits (MLF + MLT), Mandarin, with MOOC.
- 2021 Fall and 2023 Spring: 3 credits, English.
- 2023 Fall: 3 credits, Mandarin, with MOOC offline and physical discussions stimulated by ChatGPT.
- 2024: 3 credits, English, TAICA.
- 2026: 3 credits, Mandarin, TAICA, with MOOC offline and discussions stimulated by ChatGPT.

The reason for moving the lectures offline is that many students prefer watching a recording on their own time, which frees the precious class time to encourage speculation rather than lecturing. Three things are new this semester.

Cross-university teaching (2nd time). Taiwan needs more AI talent, so the course accepted an invitation from TAICA (Taiwan AI College Alliance) to be offered to 50+ universities — with thanks to the MOE for more TAs, a YouTube screencast to break the time and space boundary, and some online TA hours to be planned. An attempt to further increase the impact in this new AI age.

Final exam. AI can solve all of your homework with a high success rate. That is not a totally bad thing for teaching, but it makes fair grading exponentially difficult, so we will experiment with a closed-book final exam bearing some similarity to the homework — an old-fashioned yet fair grading protocol, until something better is designed.

AI-assisted TAs. AI can answer many questions with 90%+ correctness in a friendly manner; the question is how to best help learning. We are building an AI agent to answer students' questions, guarded by the TAs' regular human checking — human-computer collaboration towards better teaching and learning.

On enrollment: per school policy, students who have taken another version of the course (ML Foundations, ML Techniques, or the 4-credit ML) are generally *not* allowed to take it again. We will not check, but the school may, and multiple takings may not look nice on a transcript anyway. NTU students sign up through the link on the course infopage, processed progressively; a limited quota is reserved for TAICA students from other universities. Auditing is welcome in general, as long as it does not interfere with official members. *Leave as soon as possible — give your classmates a chance to be miserable.*

The course is supported by sixteen teaching assistants, reachable at html_ta@csie.ntu.edu.tw.

- | | |
|-----------------------------|-------------------------|
| • Cheng-Han Hsieh (*), CSIE | • Yen-Hung Hsueh, EE |
| • Guan-Chen Chen, CSIE | • Li-Cheng Hsu, EE |
| • Chun-Yang Cheng, CSIE | • Jung-Sheng Hung, GINM |
| • Sheng-Yuan Chang, CSIE | • Yun-Shi Lee, IM |
| • Tsung-Lun Chiang, EE | • Yi-Ting Lo, CSIE |
| • Ting-Chieh Chiu, CSIE | • Han-Hwa Shih, GICE |
| • Ting-Wei Chuang, CSIE | • Yen-Chu Tsai, CSIE |
| • Kang-Yu Fan, CSIE | • Yi-Ming Yao, Math |

THE Book

Learning from Data: A Short Course

Y. Abu-Mostafa (Caltech), M. Magdon-Ismael (RPI), H.-T. Lin (NTU)

The idea was initiated in 2008. Its five chapters are closely needed for the first part of the class, and further e-Chapters are used in the second part; the teaching follows the book, with suggested reading inside it. Copies can be found at the NTU Library (a reserved copy in the shared course material area) and in R536 (shared copies to be read in the room, if open); the Chuan-Hwa Book Company has imported some copies, and Amazon/Kindle is the main selling channel in the US. If the book is not affordable to you but you really want to read it, email the instructor and he will see how he can help.

THE Principle

Taking any unfair advantage over other class members is not allowed. It is everyone's responsibility to maximize the level of fairness.

NO CHEATING
NO LYING
NO PLAGIARISM
NO PIRATING of THE BOOK

There are very serious consequences. In practice this means: homework discussions are encouraged, but write the final solutions independently and understand them fully; references — books, notes, the Internet, AI tools — can be consulted but not copied from, and you are strongly encouraged to save your AI prompts and interaction logs. There is no need to lend, borrow, sell, buy or trade a solution, a piece of code or a dialogue, and doing so is not allowed.

Grading

There is no midterm. The main reference is

homework sets	30%
final exam	30%
final project	40%

after which the raw score goes through an order-preserving normalization: a raw score of 80 with term rank A is possible, and so is a raw score of 60 with term rank F; raw scores 80 and 60 both mapping to B is possible but unlikely; raw scores 80 and 60 mapping to F and B respectively is impossible. For non-NTU students the rank will tentatively be converted by the NTU metric back to a score in $[0, 100]$, which the student's own university then converts back to a rank.

Homework uses multiple-choice problems for auto-grading, with photographed or scanned details and code uploaded for selective grading by the TAs — there is a penalty if your details are not logical — and for plagiarism detection. Late parts are worth 90% if twelve hours late, 80% if one day late, and so on; each person is granted four penalty-free late half-days, the gold medals. About a quarter to a third of the problems are programming assignments, in any language, on any platform, with any packages; AI-generated code is fine if you co-wrote it with the AI and your logs justify that.

The *final project* is a teamed in-class competition, with team size to be decided, co-run with Prof. Shou-de Lin's *Machine Learning: Theory and Practice* class — so the two classes cannot both be taken this semester. You will need to sign and abide by a data policy. It can make you rich, if you know what I mean.

Contents

1. The Learning Problem	7
1.1. Course Introduction	7
1.2. What is Machine Learning	8
1.3. Applications of Machine Learning	10
1.4. Components of Machine Learning	13
1.5. Machine Learning and Other Fields	15
2. Learning to Answer Yes/No	18
2.1. Perceptron Hypothesis Set	18
2.2. Perceptron Learning Algorithm (PLA)	21
2.3. Guarantee of PLA	24
2.4. Non-Separable Data	28
3. Types of Learning	31
3.1. Learning with Different Output Space $\mathcal{Y}$	31
3.2. Learning with Different Data Label y_n	34
3.3. Learning with Different Protocol $f \Rightarrow (\mathbf{x}_n, y_n)$	37
3.4. Learning with Different Input Space $\mathcal{X}$	39

1. The Learning Problem

Lecture 1

§1.1. Course Introduction

9 Sep.

— slides p.1–5 —

Machine learning is a mixture of theoretical and practical tools, so any course about it has to decide how to mix them. There are two natural extremes, and both of them fail in their own way.

- *Theory oriented*: derive everything deeply, so that every statement rests on a proof and the understanding is solid. The cost is that it is less interesting to a general audience — one may finish such a course able to prove a generalization bound, yet unable to fit a model to a real data set.
- *Techniques oriented*: flash over the sexiest techniques broadly, so that the coverage looks shiny. The cost is that there are simply too many techniques: they become hard to choose between, and hard to use properly, because nothing tells you *why* one of them should work here and not there.

Our approach: foundation oriented.

By *foundation* we mean the part that does not change when the fashionable technique changes. Concretely, the course is a mixture of philosophical illustrations, key theory, core techniques, usage in practice, and hopefully jokes; the selecting principle is always *what every machine learning user should know*. The payoff of insisting on foundations is that they transfer: a student who owns them can pick up a future or untaught technique, or go read deeper theory, without being taught each one explicitly.

The material is organized **story-like**, as a sequence of questions, each one asked only after the previous one has been answered:

1. **When** can machines learn? (illustrative + technical)
2. **Why** can machines learn? (theoretical + illustrative)
3. **How** can machines learn? (technical + practical)
4. **How** can machines learn **better**? (practical + theoretical)

Note 1.1.1. The four questions above are the backbone of the *foundations* half of the story. The NTU version continues it with three more chapters, each one a different answer to “where do good features come from”:

5. How can machines learn by *embedding numerous* features? (support vector machines)
6. How can machines learn by *combining predictive* features? (aggregation, boosting, trees and forests)
7. How can machines learn by *distilling hidden* features? (neural networks and deep learning)

This first chapter sits entirely inside question 1, and it is the most basic piece of it: before asking when machines *can* learn, we must say what the learning problem even *is*. That is the plan of the chapter: we first ask what machine learning is (Section 1.2), then look at where it already lives (Section 1.3), then cut it into its formal components (Section 1.4), and finally place it next to the neighbouring fields it is often confused with (Section 1.5).

Exercise 1.1.2 (Fun Time). Which of the following descriptions of this course is true?

- (1) the course will be taught in Taiwanese;
- (2) the course will tell me the techniques that create the android Lieutenant Commander Data in *Star Trek*;
- (3) the course will be 15 weeks long;
- (4) the course will be story-like.

Reference Answer. (4). As for the others: (1) no, the instructor's Taiwanese is unfortunately not good enough for teaching (yet); (2) no, although what we teach may serve as foundations of those (future) techniques; (3) no, unless you choose to join the next course. So: let us begin the story. ■

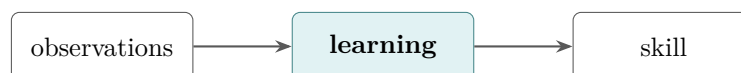
§1.2. What is Machine Learning

— slides p.6–11 —

§1.2.i. From Learning to Machine Learning

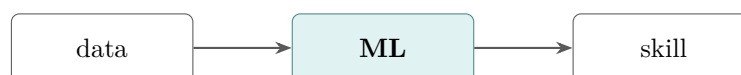
The safest way into a definition is by analogy with the thing we already do ourselves.

Definition 1.2.1 (Learning). **Learning** is acquiring *skill* with experience accumulated from *observations*.



A machine has no eyes and no childhood, so the only observations it can accumulate are the ones we hand it in recorded form. Replacing *observations* by *data* and *learning* by *computation* gives the machine version.

Definition 1.2.2 (Machine learning, first attempt). **Machine learning** is acquiring skill with experience accumulated/computed from *data*.



The word *data* is concrete enough to be useful. The word *skill* is not: it is exactly the kind of word that cannot be handed to a computer, so the definition is not finished yet.

Question 1.2.3. What is skill?

§1.2.ii. A More Concrete Definition

Intuition . skill $\Leftrightarrow$ improve some *performance measure* (e.g. prediction accuracy).

This is the move that makes the whole subject possible: a vague goal (“be good at recognizing trees”) is replaced by a number that can be computed and watched (“fraction of tree images classified correctly”). Every later chapter is, in one way or another, an argument about which number to watch and what watching it really guarantees.

Definition 1.2.4 (Machine learning). **Machine learning** is improving some *performance measure* with experience computed from *data*.



Example 1.2.5 (An application in computational finance). Take the data to be historical stock data, and the performance measure to be investment gain. Then machine learning here means: use the stock data to compute a trading rule whose gain is higher than what we had before.



§1.2.iii. Why Use Machine Learning?

We now know what machine learning *is*. A separate matter is why one would ever want it, given that a computer can already be programmed directly.

Question 1.2.6. Why use machine learning?

Example 1.2.7 (Tree recognition). Suppose we want a system that looks at a photograph and answers whether there is a tree in it.

- To ‘*define*’ *trees and hand-program* the answer is difficult: try writing down, in code, a condition that accepts every pine, palm and bonsai and rejects every bush and lamppost.
- To *learn from data (observations) and recognize* is easy enough that a 3-year-old can do it, having never received a definition of “tree” in their life — only examples.

So an ‘ML-based tree recognition system’ can be easier to *build* than a hand-programmed one, even though it is harder to *explain*.

ML is an alternative route to build complicated systems.

The useful way to read that moral is as a list of situations where the direct route is blocked:

- when humans cannot program the system manually — navigating on Mars;
- when humans cannot ‘define the solution’ easily — speech/visual recognition;
- when rapid decisions are needed that humans cannot make — high-frequency trading;
- when the system must be user-oriented at a massive scale — consumer-targeted marketing.

Remark 1.2.8. Give a computer a fish, you feed it for a day; teach it how to fish, you feed it for a lifetime. :-)

§1.2.iv. Key Essence of Machine Learning

Machine learning is not always the right tool, and the definition above already tells us when it is not. Reading [Definition 1.2.4](#) backwards, three things must hold before the phrase even makes sense.

1. There exists some **underlying pattern** to be learned — so that a ‘performance measure’ *can* be improved. Without a pattern there is nothing to be better at.
2. But there is no programmable (easy) *definition* of that pattern — so that ‘ML’ is *needed*. If you can just write the rule down, write it down.
3. Somehow there is **data** about the pattern — so that ML has some ‘inputs’ to learn from. A pattern nobody has recorded cannot be computed from.

Key essence: it helps decide whether to use ML at all.

Exercise 1.2.9 (Fun Time). Which of the following is best suited for machine learning?

- (1) predicting whether the next cry of a baby girl happens at an even-numbered minute or not;
- (2) determining whether a given graph contains a cycle;
- (3) deciding whether to approve a credit card to some customer;
- (4) guessing whether the earth will be destroyed by the misuse of nuclear power in the next ten years.

Reference Answer. (3), and the three essences say exactly why each of the others fails.

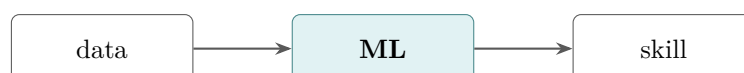
- (1) Fails essence 1: there is no pattern.
- (2) Fails essence 2: the definition is programmable (run a cycle detection algorithm).
- (3) Passes all three — pattern: customer behaviour; definition: not easily programmable; data: the history of bank operations.
- (4) Fails essence 3: there is arguably no (or not enough) data yet.

■

§1.3. Applications of Machine Learning

— slides p.12–16 —

Every application below is the same picture,



with the two ends filled in differently. So the exercise in reading an application is always: *what is the data, and what is the skill?*

§1.3.i. Daily Needs: Food, Clothing, Housing, Transportation

1. **Food** (Sadilek et al., 2013). *Data*: Twitter data (words + location). *Skill*: tell the food-poisoning likeliness of a restaurant properly.
2. **Clothing** (Abu-Mostafa, 2012). *Data*: sales figures + client surveys. *Skill*: give good fashion recommendations to clients.
3. **Housing** (Tsanas and Xifara, 2012). *Data*: characteristics of buildings and their energy load. *Skill*: predict the energy load of other buildings closely.
4. **Transportation** (Stallkamp et al., 2012). *Data*: some traffic sign images and their meanings. *Skill*: recognize traffic signs accurately.

ML is everywhere!

§1.3.ii. Education

Data: students' records on quizzes in a mathematics tutoring system. *Skill*: predict whether a student can give a correct answer to *another* quiz question.

What makes this example instructive is that the pattern one guesses is not a formula in the recorded variables at all; it is a formula in quantities nobody ever measured:

$$\text{answer correctly} \approx \llbracket \text{recent strength of student} > \text{difficulty of question} \rrbracket.$$

Notation . $\llbracket \cdot \rrbracket$ is the *Iverson bracket*: it equals 1 when the statement inside is true and 0 when it is false.

Neither “strength” nor “difficulty” appears in the data — only who answered what, and whether it was right. Feeding roughly 9 million records from 3000 students to a learning algorithm lets it *reverse-engineer* both numbers automatically, purely from the requirement that the formula above should reproduce the observed answers.

This was the key part of the world-champion system from National Taiwan University in KDDCup 2010.

§1.3.iii. Entertainment: Recommender Systems

Data: how many users have rated some movies. *Skill*: predict how a user would rate an unrated movie.

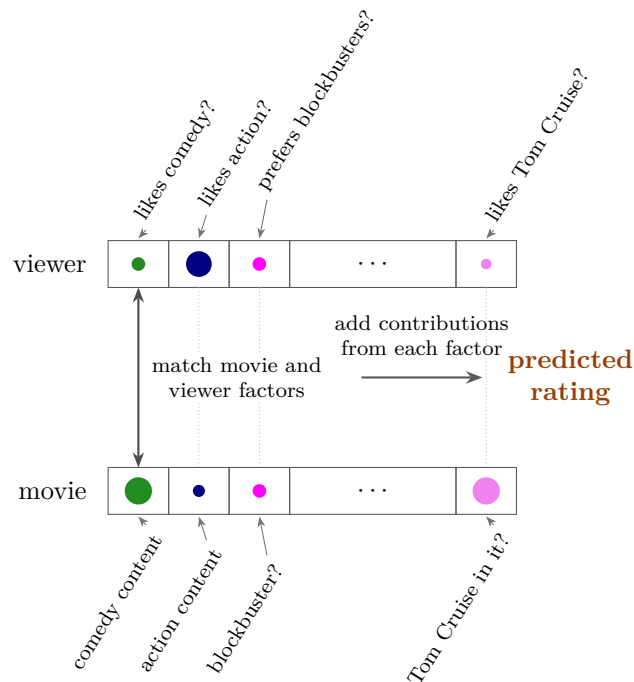
Example 1.3.1 (A hot problem). • A competition held by Netflix in 2006: 100,480,507 ratings that 480,189 users gave to 17,770 movies, with a 10% improvement worth a one-million-dollar prize.

- A similar competition (movies $\rightarrow$ songs) held by Yahoo! in KDDCup 2011: 252,800,275 ratings that 1,000,990 users gave to 624,961 songs.

Question 1.3.2. How can machines learn our preferences?

The same trick as in the education example applies, and it is worth seeing that it is literally the same trick. Postulate that a viewer and a movie are each described by the *same* list of hidden **factors** — how much comedy, how much action, how much of a blockbuster — read on

the viewer's side as *how much this viewer wants it* and on the movie's side as *how much this movie has it*.



Both rows carry the same factors in the same order, so a viewer and a movie live in one common space; the size of a dot is the size of a number. The vertical arrow pairs slot k with slot k , the horizontal one sums the pairs, and together they are a single formula: writing v_k for how much the viewer wants factor k and m_k for how much the movie has of it,

$$\text{predicted rating} \approx \sum_{k=1}^K v_k m_k,$$

the inner product of the two factor vectors. The product is the point — a factor contributes only where the two sides *agree*, so our action-loving viewer would rate this comedy rather low.

The dashes say the list is long, and the last slot says how arbitrary it may be. Nobody drew that list up: as with the students' strength and difficulty, the factors are never observed, and the learning is what invents them.

pattern: rating $\leftarrow$ viewer/movie factors;

learning: known ratings $\rightarrow$ learned factors $\rightarrow$ unknown rating prediction.

Again the key part of the world-champion system from National Taiwan University, this time in KDDCup 2011.

Exercise 1.3.3 (Fun Time). Which of the following fields *cannot* use machine learning?

- (1) Finance;
- (2) Medicine;
- (3) Law;

(4) none of the above.

Reference Answer. (4). Finance can predict stock prices from data, medicine can predict medicine effects from data, and law can summarize legal documents from data. Welcome to study this hot topic. ■

§1.4. Components of Machine Learning

— slides p.17–22 —

So far “machine learning” has been a box in a picture. This section opens the box and names its parts. The running metaphor is *credit approval*: a bank receives an application and must decide whether issuing a card to this person is good for the bank.

Applicant Information	
age	23 years
gender	female
annual salary	NTD 1,000,000
year in residence	1 year
year in job	0.5 year
current debt	200,000

The unknown pattern to be learned is: *is approving this credit card good for the bank?*

§1.4.i. Formalizing the Learning Problem

Notation .

- **Input:** $x \in \mathcal{X}$ (a customer application).
- **Output:** $y \in \mathcal{Y}$ (good/bad after approving the credit card).
- The unknown pattern to be learned $\Leftrightarrow$ the **target function**

$$f: \mathcal{X} \rightarrow \mathcal{Y}$$

(the ideal credit approval formula).

- The data $\Leftrightarrow$ the **training examples**

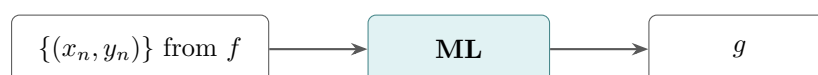
$$\mathcal{D} = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

(historical records in the bank).

- The skill, hopefully with good performance, $\Leftrightarrow$ the **hypothesis**

$$g: \mathcal{X} \rightarrow \mathcal{Y}$$

(the ‘learned’ formula to be used).

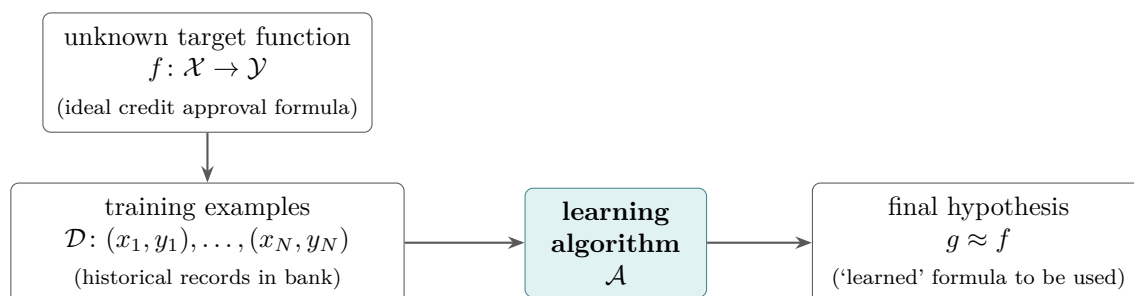


Remark 1.4.1. The asymmetry between f and g is the whole difficulty of the subject, so it is worth stating plainly.

- f is *unknown* — indeed, by essence 2 of the previous section, it has no programmable definition. We never see f ; we only ever see N of its values.
- g is what we produce, and we hope $g \approx f$. But g may differ from f , and perfection is ‘impossible’ precisely because f is unknown — there is no way to check the answer.

Everything in *Why can machines learn?* is an attempt to say something rigorous about $g \approx f$ despite never seeing f .

Drawing the arrows in the order in which things actually happen gives the learning flow.



§1.4.ii. The Learning Model

The diagram still hides one thing.

Question 1.4.2. What does g look like?

An algorithm cannot search over *all* functions $\mathcal{X} \rightarrow \mathcal{Y}$, so we must first decide which formulas are even candidates.

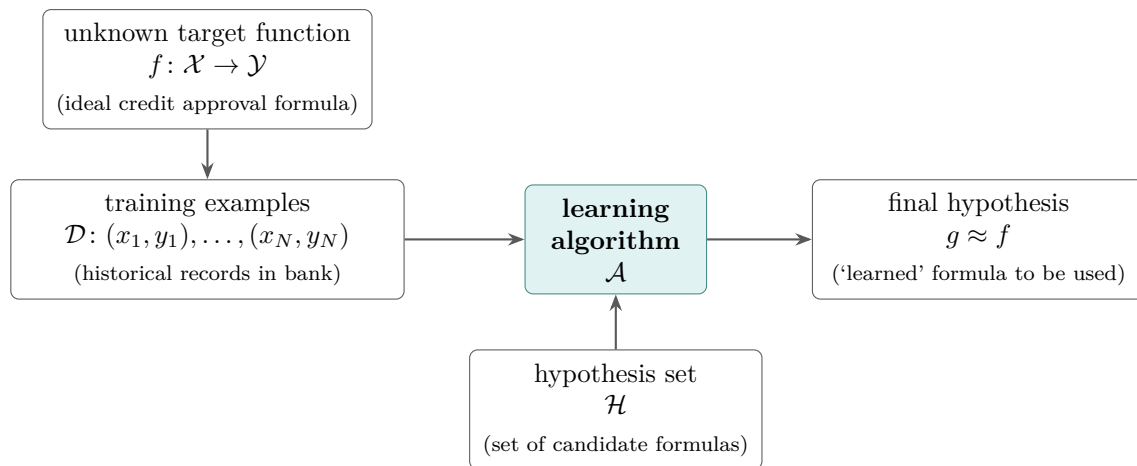
Definition 1.4.3 (Hypothesis set). The **hypothesis set** $\mathcal{H} = \{h_k\}$ is the set of candidate formulas that the algorithm is allowed to return, so that $g \in \mathcal{H}$.

Example 1.4.4. For credit approval, one might approve if

- h_1 : annual salary $>$ NTD 800,000;
- h_2 : debt $>$ NTD 100,000 (really?);
- h_3 : year in job ≤ 2 (really?).

Note that $\mathcal{H}$ happily contains bad hypotheses next to good ones. That is not a defect: it is up to $\mathcal{A}$ to pick the ‘best’ one out of $\mathcal{H}$ and call it g .

Definition 1.4.5 (Learning model). A **learning model** is the pair consisting of the hypothesis set $\mathcal{H}$ and the learning algorithm $\mathcal{A}$.



This is the picture the rest of the course elaborates. The dashed halves of the story are already visible in it: f and $\mathcal{D}$ are given by the world and we cannot choose them, while $\mathcal{H}$ and $\mathcal{A}$ are ours to design.

Machine learning: use data to compute a hypothesis g that approximates the target f .

Exercise 1.4.6 (Fun Time). How should the four sets below be used to form a learning problem for song recommendation?

- $S_1 = [0, 100]$;
- $S_2 =$ all possible (userid, songid) pairs;
- $S_3 =$ all formulas that ‘multiply’ user factors and song factors,
indexed by all possible combinations of such factors;
- $S_4 =$ 1,000,000 pairs of ((userid, songid), rating).

- (1) $S_1 = \mathcal{X}$, $S_2 = \mathcal{Y}$, $S_3 = \mathcal{H}$, $S_4 = \mathcal{D}$;
- (2) $S_1 = \mathcal{Y}$, $S_2 = \mathcal{X}$, $S_3 = \mathcal{H}$, $S_4 = \mathcal{D}$;
- (3) $S_1 = \mathcal{D}$, $S_2 = \mathcal{H}$, $S_3 = \mathcal{Y}$, $S_4 = \mathcal{X}$;
- (4) $S_1 = \mathcal{X}$, $S_2 = \mathcal{D}$, $S_3 = \mathcal{Y}$, $S_4 = \mathcal{H}$.

Reference Answer. (2). Reading the whole problem in one line,

$$S_4 \xrightarrow{\mathcal{A} \text{ on } S_3} (g: S_2 \rightarrow S_1),$$

i.e. the input space is the set of (user, song) pairs, the output space is the rating scale $[0, 100]$, the hypothesis set is the family of factor-multiplying formulas, and the data is the list of observed ratings. ■

§1.5. Machine Learning and Other Fields

— slides p.23–27 —

Machine learning did not grow up alone, and much of the confusion around the name comes from its neighbours. Comparing the one-line definitions is enough to see both the overlap and the difference in *focus* — which is usually where the real difference lies.

§1.5.i. Machine Learning and Data Mining

Machine Learning	Data Mining
use data to compute a hypothesis g that approximates a target f	use (huge) data to find a property that is <i>interesting</i>

- If the ‘interesting property’ *is* the ‘hypothesis that approximates the target’, then ML = DM. This is usually what KDDCup does.
- If the ‘interesting property’ is merely *related* to such a hypothesis, then DM can help ML and vice versa — often, but not always.
- Traditional DM also puts weight on efficient computation in large databases, which ML traditionally does not.

It is difficult to distinguish ML and DM in reality.

§1.5.ii. Machine Learning and Artificial Intelligence

Machine Learning	Artificial Intelligence
use data to compute a hypothesis g that approximates a target f	compute something that shows <i>intelligent behaviour</i>

- $g \approx f$ is something that shows intelligent behaviour, so ML can realize AI — among other routes.
- For example, in chess playing: traditional AI builds a game tree, whereas ML for AI learns from board data.

ML is one possible route to realize AI.

§1.5.iii. Machine Learning and Statistics

Machine Learning	Statistics
use data to compute a hypothesis g that approximates a target f	use data to make <i>inference</i> about an unknown process

- g is an inference outcome and f is something unknown, so statistics can be used to achieve ML.
- Traditional statistics also focuses on provable results under mathematical assumptions, and cares less about computation.

Statistics: many useful tools for ML.

Exercise 1.5.1 (Fun Time). Which of the following claims is *not* totally true?

- (1) machine learning is a route to realize artificial intelligence;
- (2) machine learning, data mining and statistics all need data;
- (3) data mining is just another name for machine learning;

(4) statistics can be used for data mining.

Reference Answer. (3). While data mining and machine learning do share a huge overlap, they are arguably not equivalent, because of the difference of focus. ■

Summary

Course Introduction foundation oriented and story-like.

What is Machine Learning use data to approximate a target.

Applications of Machine Learning almost everywhere.

Components of Machine Learning $\mathcal{A}$ takes $\mathcal{D}$ and $\mathcal{H}$ to get g .

Machine Learning and Other Fields related to DM, AI and statistics.

We have a frame, $(\mathcal{X}, \mathcal{Y}, f, \mathcal{D}, \mathcal{H}, \mathcal{A}, g)$, but not a single concrete learning model to put in it: we never said what any $h \in \mathcal{H}$ actually computes, nor how $\mathcal{A}$ searches. The next chapter fills in the simplest useful choice of $\mathcal{H}$ and $\mathcal{A}$, for the simplest useful output space $\mathcal{Y} = \{-1, +1\}$ — that is, learning to answer yes/no.

2. Learning to Answer Yes/No

Lecture 2

As previously seen, [Section 1.4](#) ended with a complete frame for the learning problem: an algorithm $\mathcal{A}$ takes the data $\mathcal{D}$ and a hypothesis set $\mathcal{H}$, and returns a final hypothesis g which we hope satisfies $g \approx f$. Nothing in that frame said what a single $h \in \mathcal{H}$ computes, nor how $\mathcal{A}$ searches.

16 Sep.

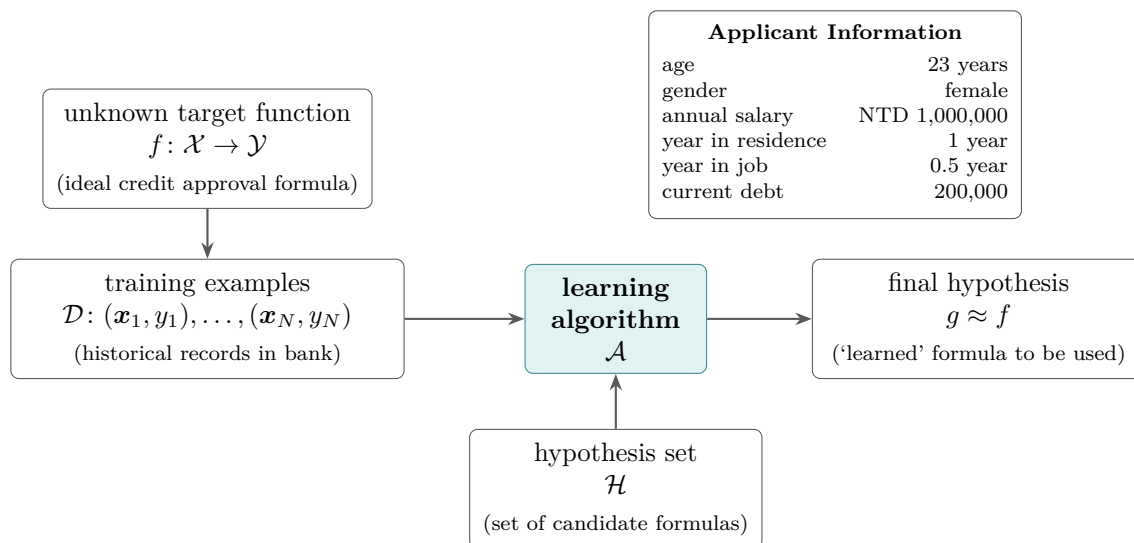
This chapter fills that gap in the simplest interesting case, where the output is a yes/no answer, and gets both an $\mathcal{H}$ and an $\mathcal{A}$ out of it — our first concrete learning model.

§2.1. Perceptron Hypothesis Set

— slides p.2–6 —

§2.1.i. The Credit Approval Problem Revisited

Put the credit approval problem of [Section 1.4](#) back into the flow, this time with the applicant record written out.



Question 2.1.1. What hypothesis set can we use?

§2.1.ii. Features of a Customer

The bank holds a customer record of d numbers,

$$\mathbf{x} = (x_1, x_2, \dots, x_d),$$

the **features** of the customer — age, annual salary, year in job, current debt, and so on — and it must answer yes or no.

Notation . From this chapter on, an input $\mathbf{x}$ is a vector of d real features, written in bold. In [Section 1.4](#) the input was an abstract element of $\mathcal{X}$; giving it coordinates is exactly the step that lets us write down formulas for h .

§2.1.iii. The Perceptron

age	23 years
annual salary	NTD 1,000,000
year in job	0.5 year
current debt	200,000

The simplest thing one can do with d numbers is to weigh them against each other. Give each feature x_i an importance w_i , form a weighted *score* $\sum_{i=1}^d w_i x_i$, and compare it against a threshold:

$$\begin{aligned} \text{approve credit if } & \sum_{i=1}^d w_i x_i > \text{threshold;} \\ \text{deny credit if } & \sum_{i=1}^d w_i x_i < \text{threshold.} \end{aligned}$$

A positive w_i means the feature speaks in the customer's favour (annual salary), a negative one means it speaks against (current debt), and a large $|w_i|$ means the feature matters a lot.

Since the answer is yes or no, take

$$\mathcal{Y} = \{+1 \text{ (good), } -1 \text{ (bad)}\},$$

and ignore the boundary case of a score exactly equal to the threshold, i.e. treat 0 as ignored. Then the rule above is a single formula.

Definition 2.1.2 (Perceptron). For a weight vector $w_1, \dots, w_d$ and a threshold, the hypothesis

$$h(\mathbf{x}) = \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) - \text{threshold} \right)$$

is called a **perceptron**, for historical reasons. The hypothesis set $\mathcal{H}$ of this chapter is the set of all such h , i.e. one h for every choice of weights and threshold.

§2.1.iv. Vector Form

The threshold is an eyesore: it is a parameter of the same kind as the weights, but it sits outside the sum. The standard repair is to absorb it as the weight of a constant feature:

$$\begin{aligned} h(\mathbf{x}) &= \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) - \text{threshold} \right) \\ &= \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) + \underbrace{(-\text{threshold})}_{w_0} \cdot \underbrace{(+1)}_{x_0} \right) \\ &= \text{sign} \left(\sum_{i=0}^d w_i x_i \right) \\ &= \text{sign} \left(\mathbf{w}^\top \mathbf{x} \right). \end{aligned}$$

Notation . From now on $\mathbf{w} = (w_0, w_1, \dots, w_d)^\top$ and $\mathbf{x} = (x_0, x_1, \dots, x_d)^\top$ are the ‘tall’ versions, with $x_0 \equiv 1$ always. Each tall $\mathbf{w}$ is a hypothesis h , so from here on we say ‘the hypothesis $\mathbf{w}$ ’ and mean $h(\mathbf{x}) = \text{sign}(\mathbf{w}^\top \mathbf{x})$.

The gain is not only cosmetic: everything below manipulates $\mathbf{w}$ as a vector, and a stray additive constant would have to be carried through every one of those manipulations.

§2.1.v. Perceptrons in $\mathbb{R}^2$

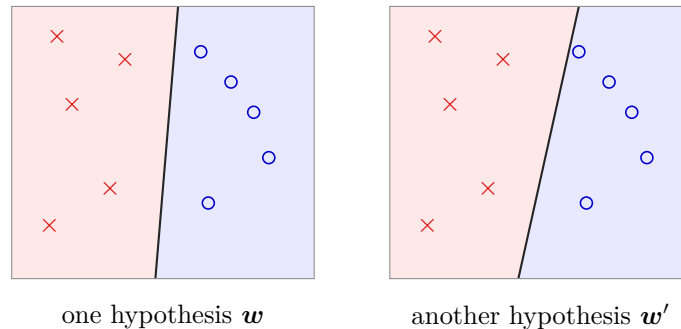
Question 2.1.3. What do perceptrons h ‘look like’?

Take $d = 2$, so that

$$h(\mathbf{x}) = \text{sign}(w_0 + w_1x_1 + w_2x_2).$$

The set where the score vanishes, $w_0 + w_1x_1 + w_2x_2 = 0$, is a line; h says $+1$ on one side of it and -1 on the other. So:

- customer features $\mathbf{x}$ are points on the plane (or points in $\mathbb{R}^d$);
- labels y are drawn as $\circ$ for $+1$ and $\times$ for -1 ;
- a hypothesis h is a line (or a hyperplane in $\mathbb{R}^d$), positive on one side and negative on the other;
- a different line classifies the customers differently.



perceptrons $\Leftrightarrow$ linear (binary) classifiers.

Exercise 2.1.4 (Fun Time). Consider using a perceptron to detect spam messages. Assume that each email is represented by the frequency of keyword occurrence, and that an output of $+1$ indicates spam. Which keywords below should have large *positive* weights in a good perceptron for the task?

- (1) coffee, tea, hamburger, steak;
- (2) free, drug, fantastic, deal;
- (3) machine, learning, statistics, textbook;
- (4) national, Taiwan, university, coursera.

Reference Answer. (2). The occurrence of a keyword with a positive weight increases the ‘spam score’, and hence those keywords should be the ones that often appear in spam. ■

§2.2. Perceptron Learning Algorithm (PLA)

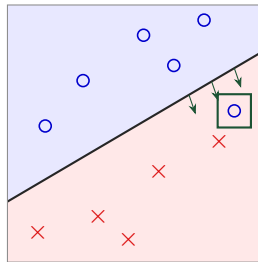
— slides p.7–12 —

Half of the learning model is still missing.

Question 2.2.1. $\mathcal{H}$ = all possible perceptrons, $g = ?$

§2.2.i. Selecting g from $\mathcal{H}$

- What we *want* is $g \approx f$. This is hard to even check, because f is unknown.
- What is *almost necessary* is $g \approx f$ on $\mathcal{D}$, ideally $g(\mathbf{x}_n) = f(\mathbf{x}_n) = y_n$ for every n . This we can check, because $\mathcal{D}$ is in front of us.
- What makes it *difficult* is that $\mathcal{H}$ is of infinite size, so we cannot look at the candidates one by one.
- The *idea* that resolves this: start from some g_0 , and ‘correct’ its mistakes on $\mathcal{D}$.



Infinitely many candidates are no obstacle if we never enumerate them, but instead walk from one to the next. By the previous subsection we may represent g_0 by its weight vector $\mathbf{w}_0$, so the walk happens in $\mathbb{R}^{d+1}$ rather than in the abstract set $\mathcal{H}$.

§2.2.ii. The Algorithm

Algorithm 2.1: Perceptron Learning Algorithm (PLA)

Input: training examples $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$

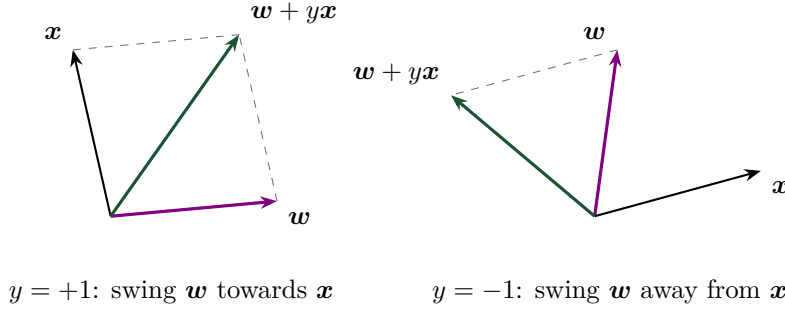
Output: $\mathbf{w}_{\text{PLA}}$, returned as g

```

1  $\mathbf{w}_0 \leftarrow$  some initial vector, say  $\mathbf{0}$ ;
2 for  $t = 0, 1, 2, \dots$  do
3   if  $\mathbf{w}_t$  makes no mistake on  $\mathcal{D}$  then return  $\mathbf{w}_t$  as  $\mathbf{w}_{\text{PLA}}$ ;
4   find a mistake of  $\mathbf{w}_t$ , called  $(\mathbf{x}_{n(t)}, y_{n(t)})$ , i.e. one with  $\text{sign}(\mathbf{w}_t^\top \mathbf{x}_{n(t)}) \neq y_{n(t)}$ ;
5   (try to) correct the mistake by  $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + y_{n(t)} \mathbf{x}_{n(t)}$ ;
```

That is it — a fault confessed is half redressed. :-)

The update rule looks arbitrary until one draws it. A mistake at $(\mathbf{x}, y)$ with $y = +1$ means $\mathbf{w}^\top \mathbf{x} < 0$, i.e. $\mathbf{w}$ and $\mathbf{x}$ point more than 90° apart; adding $\mathbf{x}$ swings $\mathbf{w}$ *towards* $\mathbf{x}$. A mistake with $y = -1$ means $\mathbf{w}^\top \mathbf{x} > 0$; subtracting $\mathbf{x}$ swings $\mathbf{w}$ *away* from $\mathbf{x}$. Either way the correction is $\mathbf{w} + y\mathbf{x}$.



§2.2.iii. Practical Implementation of PLA

algorithm 2.1 says “find a mistake” without saying *which* one, and a literal implementation would rescan $\mathcal{D}$ from the beginning after every correction. Cyclic PLA removes both vaguenesses at once: fix an order on the examples, keep a cursor into it, and let “the next mistake” mean *the next mistake the cursor runs into*, continuing from where the last correction happened and wrapping around at the end of the list.

Written out, with $\pi(1), \dots, \pi(N)$ the chosen order:

Algorithm 2.2: Cyclic PLA

Input: training examples $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$ and a fixed order $\pi(1), \dots, \pi(N)$ of $1, \dots, N$

Output: w_{PLA} , returned as g

```

1  $w \leftarrow 0$ ;
2  $i \leftarrow 1$ ;                                // cursor: where the next scan resumes
3  $c \leftarrow 0$ ;                                // examples seen in a row with no mistake
4 while  $c < N$  do
5    $n \leftarrow \pi(i)$ ;
6   if  $\text{sign}(w^\top x_n) \neq y_n$  then
7      $w \leftarrow w + y_n x_n$ ;                 // a mistake: correct it
8      $c \leftarrow 0$ ;                           // the clean run is broken
9   else
10     $c \leftarrow c + 1$ ;                         // this one is already right
11     $i \leftarrow (i \bmod N) + 1$ ;                 // advance, wrapping around
12 return  $w$  as  $w_{\text{PLA}}$ ;
```

Two points deserve spelling out.

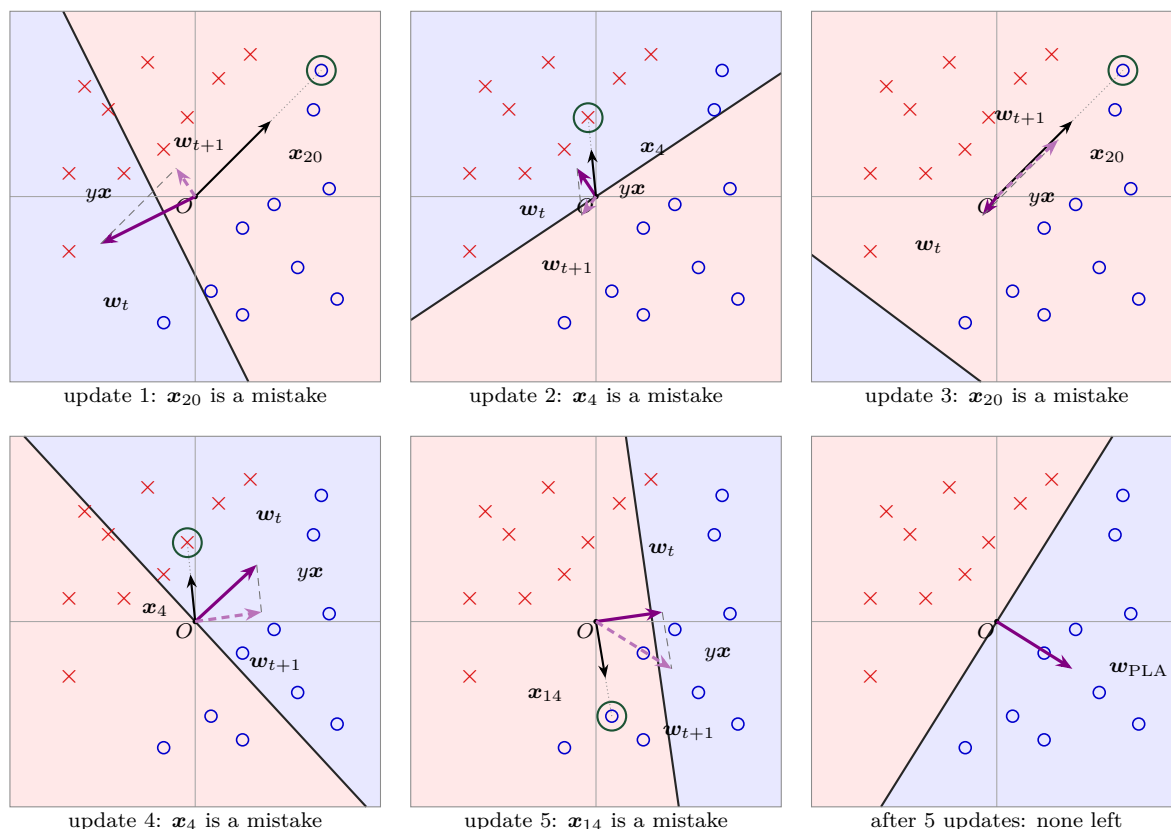
- **When does the loop end?** When c reaches N , i.e. when N examples in a row have been checked and none of them was a mistake. Since the cursor moves cyclically, any N consecutive visits run through every example exactly once — so $c = N$ says precisely that a full cycle passed without encountering a mistake, which is to say that the current w is correct on all of $\mathcal{D}$. There is no need for the scan to be aligned with the start of the list.
- **Why reset c to 0 after every correction?** Because a correction changes w , and the new w may well be wrong on examples the cursor has already walked past — exactly what happens to x_{15} in the run below. Everything checked before the correction was checked against a *different* hypothesis and no longer counts, so the tally has to start over.

The order π may be the naïve cycle $1, \dots, N$, or a precomputed random cycle. Note also that the loop counter here is not the t of algorithm 2.1: this loop turns once per example *examined*, whereas t — and the T bounded in Theorem 2.3.6 — counts only the examples *actually corrected*.

§2.2.iv. Seeing is Believing

Run it on a small separable data set of $N = 20$ examples. Each frame shows the hypothesis $\mathbf{w}_t$ that PLA currently holds — its line, and the two shaded sides it calls +1 (blue) and -1 (red) — together with the example $\mathbf{x}_{n(t)}$ it still gets wrong, circled. A mistake is visible as a $\circ$ sitting in the red half or a $\times$ sitting in the blue half.

Every frame is drawn with the origin O at its centre, so that all three vectors can be read off from there: the circled $\mathbf{x}_{n(t)}$ (black), the current $\mathbf{w}_t$ (solid, the one whose line is shaded) and the $\mathbf{w}_{t+1}$ the correction is about to produce (dashed) — which is the line drawn in the next frame. All three are at 0.6 of their true length so that they stay beside the data; the dotted continuation of $\mathbf{x}_{n(t)}$ runs on to the example itself.



Because the three share one scale, the dashed segment joining the two weight tips is the step $y_{n(t)}\mathbf{x}_{n(t)}$ — parallel to $\mathbf{x}_{n(t)}$ and of the same drawn length — so the update rule is literally the triangle that closes:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + y_{n(t)}\mathbf{x}_{n(t)}.$$

The run is started from an initial $\mathbf{w}_0$ that is merely a guess — taking $\mathbf{w}_0 = \mathbf{0}$, as [algorithm 2.1](#) suggests, would leave the first frame with no line to look at — so already the first frame shows a genuine classifier, and a genuinely bad one. Each $\mathbf{w}_t$ is perpendicular to its own line and points into the +1 side, so watch it swing towards a circled $\circ$ and away from a circled $\times$. A correction made for one example happily breaks another: here $\mathbf{x}_{20}$ has to be corrected twice, in updates 1 and 3, and so does $\mathbf{x}_4$, in updates 2 and 4. After five corrections no mistake is left and the algorithm stops.

It works like a charm in fewer than 20 lines of code. But watching it work once is not a guarantee, and two kinds of issue are left open.

Question 2.2.2 (Some remaining issues of PLA). **Algorithmic:** does it *halt*, with no mistake?

- naïve cyclic: ??
- random cyclic: ??
- other variant: ??

Learning: is $g \approx f$?

- on $\mathcal{D}$, if it halts: yes, since then there is no mistake;
- outside $\mathcal{D}$: ??
- if not halting: ??

To be shown: if $(\dots)$, then after ‘enough’ corrections any PLA variant halts.

The rest of the chapter settles the first question and postpones the “outside $\mathcal{D}$ ” half of the second one to *Why can machines learn?*

Exercise 2.2.3 (Fun Time). Let us try to think about why PLA may work. Let $n = n(t)$. According to the rule

$$\text{sign}(\mathbf{w}_t^\top \mathbf{x}_n) \neq y_n, \quad \mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + y_n \mathbf{x}_n,$$

which formula is true?

- (1) $\mathbf{w}_{t+1}^\top \mathbf{x}_n = y_n$;
- (2) $\text{sign}(\mathbf{w}_{t+1}^\top \mathbf{x}_n) = y_n$;
- (3) $y_n \mathbf{w}_{t+1}^\top \mathbf{x}_n \geq y_n \mathbf{w}_t^\top \mathbf{x}_n$;
- (4) $y_n \mathbf{w}_{t+1}^\top \mathbf{x}_n < y_n \mathbf{w}_t^\top \mathbf{x}_n$.

Reference Answer. (3). Simply multiply the update rule by $y_n \mathbf{x}_n$:

$$y_n \mathbf{w}_{t+1}^\top \mathbf{x}_n = y_n \mathbf{w}_t^\top \mathbf{x}_n + y_n^2 \|\mathbf{x}_n\|^2 \geq y_n \mathbf{w}_t^\top \mathbf{x}_n,$$

since $y_n^2 = 1$. Note that (1) and (2) are *too strong*: one correction need not fix the very mistake it was made for. All we get is that the quantity $y_n \mathbf{w}^\top \mathbf{x}_n$, which is positive exactly when $\mathbf{x}_n$ is classified correctly, moves in the right direction. The rule somewhat ‘tries to correct the mistake’. ■

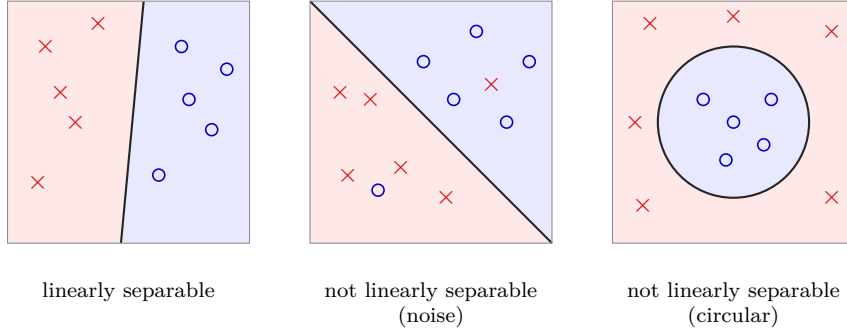
§2.3. Guarantee of PLA

— slides p.13–16 —

§2.3.i. Linear Separability

If PLA halts, then by construction there are no mistakes left, so $\mathcal{D}$ must have admitted a perfect $\mathbf{w}$ in the first place. That is a *necessary* condition for halting, and it deserves a name.

Definition 2.3.1 (Linear separability). A data set $\mathcal{D}$ is **linearly separable** if there exists a $\mathbf{w}$ that makes no mistake on it, i.e. $y_n = \text{sign}(\mathbf{w}^\top \mathbf{x}_n)$ for every n .



The converse is the real question.

Question 2.3.2. Assume a linearly separable $\mathcal{D}$. Does PLA always halt?

It does, and the proof is two facts about what a single correction does.

§2.3.ii. Two Facts

Fact 1: w_t gets more aligned with w_f

Restating Definition 2.3.1 gives the premise the whole argument runs on:

$$\mathcal{D} \text{ linearly separable} \Leftrightarrow \text{exists perfect } w_f \text{ such that } y_n = \text{sign}(w_f^\top x_n).$$

Fix such a w_f . Being **perfect** buys more than “no mistakes”: every example is not merely on the correct side of w_f ’s line but strictly *away* from it, so the worst example over the whole (finite) data set still has room to spare:

$$y_{n(t)} w_f^\top x_{n(t)} \geq \min_n y_n w_f^\top x_n > 0.$$

That strictly positive number is the entire engine of the proof, because it is paid out afresh at every single correction.

Lemma 2.3.3 ($w_f^\top w_t$ goes up)

Updating with any mistake $(x_{n(t)}, y_{n(t)})$ gives

$$w_f^\top w_{t+1} \geq w_f^\top w_t + \min_n y_n w_f^\top x_n > w_f^\top w_t.$$

Proof. Expand the update and apply the displayed bound above:

$$\begin{aligned} w_f^\top w_{t+1} &= w_f^\top (w_t + y_{n(t)} x_{n(t)}) \\ &\geq w_f^\top w_t + \min_n y_n w_f^\top x_n \\ &> w_f^\top w_t + 0. \end{aligned}$$

So w_t *appears* more aligned with w_f after an update — really? Not yet. An inner product can grow for two quite different reasons: because the angle between the two vectors shrank, which is what we want, or merely because w_t got longer, which would mean nothing at all. Lemma 2.3.3 on its own cannot tell these apart. The second fact closes exactly that hole. ■

Fact 2: w_t does not grow too fast

Here the premise is the other half of how PLA is built — it touches w_t *only* when there is a mistake to fix, and a mistake is precisely a negative agreement:

$$w_t \text{ changed only when mistake} \Leftrightarrow \text{sign}(w_t^\top x_{n(t)}) \neq y_{n(t)} \Leftrightarrow y_{n(t)} w_t^\top x_{n(t)} \leq 0.$$

Lemma 2.3.4 ($\|w_t\|^2$ is limited)

A mistake ‘limits’ the growth of $\|w_t\|^2$, even when the update uses the ‘longest’ x_n :

$$\|w_{t+1}\|^2 \leq \|w_t\|^2 + \max_n \|y_n x_n\|^2.$$

Proof. The cross term is the one to watch, and the characterisation of a mistake above says it is never positive:

$$\begin{aligned} \|w_{t+1}\|^2 &= \|w_t + y_{n(t)} x_{n(t)}\|^2 \\ &= \|w_t\|^2 + 2y_{n(t)} w_t^\top x_{n(t)} + \|y_{n(t)} x_{n(t)}\|^2 \\ &\leq \|w_t\|^2 + 0 + \|y_{n(t)} x_{n(t)}\|^2 \\ &\leq \|w_t\|^2 + \max_n \|y_n x_n\|^2. \end{aligned}$$

■

Intuition . Updating only on a mistake is what makes the algorithm work. A mistake means w_t and $y_{n(t)} x_{n(t)}$ disagree, so the correction is never a step in the direction w_t already points: it turns w_t rather than stretching it.

Now put the two together. Each fact describes a *single* correction, so apply each of them once per correction and let the bounds telescope. Start PLA from $w_0 = \mathbf{0}$ and suppose it has made T corrections.

Chaining Lemma 2.3.3 through $t = 0, 1, \dots, T-1$ adds the same positive amount T times over:

$$w_f^\top w_T \geq \underbrace{w_f^\top w_0}_{=0} + T \min_n y_n w_f^\top x_n = T \min_n y_n w_f^\top x_n,$$

so the inner product grows at least *linearly* in T . Chaining Lemma 2.3.4 the same way,

$$\|w_T\|^2 \leq \underbrace{\|w_0\|^2}_{=0} + T \max_n \|y_n x_n\|^2 = T \max_n \|x_n\|^2,$$

where $\|y_n x_n\| = \|x_n\|$ because $|y_n| = 1$. So the *squared* length grows at most linearly, which means $\|w_T\|$ itself grows at most like $\sqrt{T}$.

Dividing the first by the second is what finally answers the ‘(really?)’. The quantity

$$\frac{w_f^\top w_T}{\|w_f\| \|w_T\|}$$

is an inner product of two *unit* vectors, i.e. the cosine of the angle between w_f and w_T ; dividing by $\|w_T\|$ has removed precisely the loophole that made Lemma 2.3.3 inconclusive, so if this grows, the angle really does shrink. Feeding in the two displays — the numerator is bounded below, the denominator above — gives

$$\frac{w_f^\top w_T}{\|w_f\| \|w_T\|} \geq \frac{T \min_n y_n w_f^\top x_n}{\|w_f\| \sqrt{T \max_n \|x_n\|^2}} = \sqrt{T} \cdot \text{constant},$$

the constant being built from $\mathbf{w}_f$ and the data alone: every appearance of T has collected into the $T/\sqrt{T} = \sqrt{T}$ out front.

That is already the whole theorem. The left-hand side is a cosine and so never exceeds 1, while the right-hand side grows without bound as T does; the two can hold together only for finitely many T . PLA therefore corrects only finitely many mistakes — it halts. All that is left is to name the constant, which is what the next exercise does.

Exercise 2.3.5 (Fun Time). Let us upper-bound T , the number of mistakes that PLA ‘corrects’. Define

$$R^2 = \max_n \|\mathbf{x}_n\|^2, \quad \rho = \min_n y_n \frac{\mathbf{w}_f^\top}{\|\mathbf{w}_f\|} \mathbf{x}_n.$$

We want to show $T \leq \square$. Express the upper bound $\square$ by the two terms above.

- (1) R/ρ ;
- (2) R^2/ρ^2 ;
- (3) R/ρ^2 ;
- (4) ρ^2/R^2 .

Reference Answer. (2), and the calculation is exactly [Theorem 2.3.6](#) below: naming the two quantities turns the anonymous constant of the previous page into ρ/R , and a cosine’s ceiling of 1 then caps T at $1/(\rho/R)^2 = R^2/\rho^2$. ■

§2.3.iii. The Guarantee

Theorem 2.3.6 (PLA halts)

Let $\mathcal{D}$ be linearly separable, let $\mathbf{w}_f$ be a perfect weight vector for it, and put

$$R^2 = \max_n \|\mathbf{x}_n\|^2, \quad \rho = \min_n y_n \frac{\mathbf{w}_f^\top}{\|\mathbf{w}_f\|} \mathbf{x}_n > 0.$$

Then PLA started from $\mathbf{w}_0 = \mathbf{0}$ makes at most

$$T \leq \frac{R^2}{\rho^2}$$

corrections, and in particular it halts.

Proof. Everything has been derived already; naming R and ρ is all that is left. Note first that $\rho > 0$: it is $\min_n y_n \mathbf{w}_f^\top \mathbf{x}_n / \|\mathbf{w}_f\|$, and that minimum was shown strictly positive in Fact 1 because $\mathbf{w}_f$ is perfect. With this notation the two chained bounds read

$$\mathbf{w}_f^\top \mathbf{w}_T \geq T \min_n y_n \mathbf{w}_f^\top \mathbf{x}_n = T \rho \|\mathbf{w}_f\|, \quad \|\mathbf{w}_T\|^2 \leq T \max_n \|\mathbf{x}_n\|^2 = T R^2,$$

so that

$$\frac{\mathbf{w}_f^\top \mathbf{w}_T}{\|\mathbf{w}_f\| \|\mathbf{w}_T\|} \geq \frac{T \rho \|\mathbf{w}_f\|}{\|\mathbf{w}_f\| \sqrt{T R^2}} = \sqrt{T} \cdot \frac{\rho}{R}.$$

The anonymous constant was therefore ρ/R all along. By [Remark 2.3.7](#) the left-hand side is at most 1, so $\sqrt{T} \rho/R \leq 1$, i.e. $T \leq R^2/\rho^2$. Since the right-hand side is a finite number fixed by $\mathcal{D}$ and $\mathbf{w}_f$, PLA cannot correct mistakes forever. ■

Remark 2.3.7 (Is it really a cosine in $\mathbb{R}^{d+1}$?). We have been calling $\mathbf{w}_f^\top \mathbf{w}_T / (\|\mathbf{w}_f\| \|\mathbf{w}_T\|)$ a cosine, which is obvious for $d = 2$ but deserves a word once $\mathbf{w}$ lives in $\mathbb{R}^{d+1}$ for large d . It is still a cosine, for two reasons, and the proof needs only the weaker of them.

- **What the proof actually uses** is only that the quantity never exceeds 1, and that is the Cauchy–Schwarz inequality $|\mathbf{u}^\top \mathbf{v}| \leq \|\mathbf{u}\| \|\mathbf{v}\|$, valid in $\mathbb{R}^m$ for every m . No geometry is required.
- **Geometrically** it is a cosine in the ordinary sense too. Two vectors always span a subspace of dimension at most 2; inside that plane they make an ordinary angle, and $\mathbf{u}^\top \mathbf{v} / (\|\mathbf{u}\| \|\mathbf{v}\|)$ is its cosine. Indeed this is how the angle between two vectors is *defined* in any dimension — Cauchy–Schwarz is exactly the statement that makes the definition legitimate, by putting the ratio in $[-1, 1]$ so that an arccos exists.

So the picture of $\mathbf{w}_t$ slowly rotating towards $\mathbf{w}_f$ survives intact in any number of dimensions.

Remark 2.3.8. Both constants have a geometric reading, and the bound is more memorable through it. R is the radius of the data: the distance from the origin to the farthest example. And since $\mathbf{w}_f / \|\mathbf{w}_f\|$ is a unit normal of the perfect separating hyperplane, ρ is the smallest signed distance from an example to that hyperplane — the *margin*. So

$$T \leq \left(\frac{\text{radius of the data}}{\text{margin}} \right)^2 :$$

the more comfortably the data separates, the sooner PLA is done. This quantity returns as the central object of the chapter on support vector machines, which does not merely tolerate a large margin but goes looking for one.

§2.4. Non-Separable Data

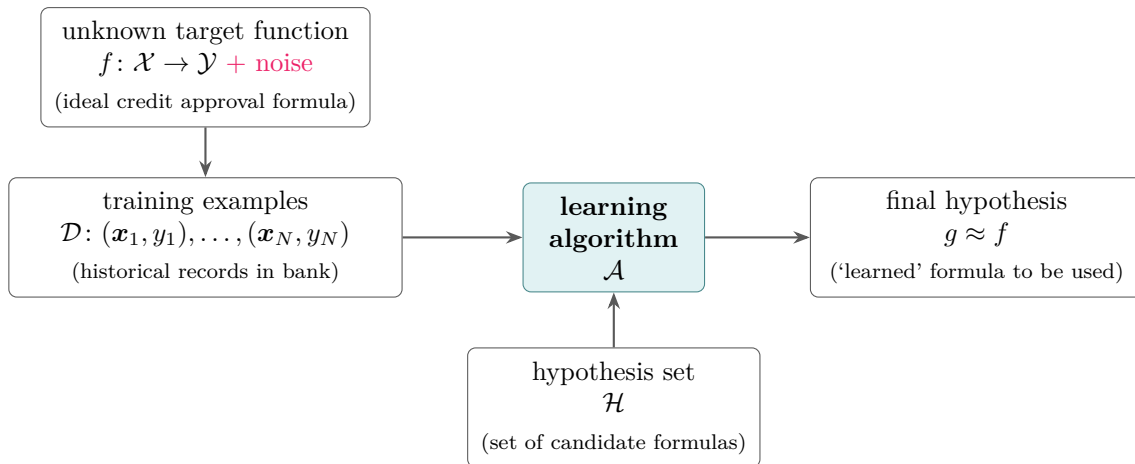
— slides p.17–22 —

§2.4.i. What We Have and What We Lack

- **Guarantee:** as long as $\mathcal{D}$ is linearly separable and we correct by mistake, the inner product of $\mathbf{w}_f$ and $\mathbf{w}_t$ grows fast while the length of $\mathbf{w}_t$ grows slowly; the PLA lines get more and more aligned with $\mathbf{w}_f$, so PLA halts.
- **Pros:** simple to implement, fast, and works in any dimension d .
- **Cons:** it *assumes* a linearly separable $\mathcal{D}$ in order to halt — and that property is unknown in advance, with the circularity that if we already knew $\mathbf{w}_f$ we would not need PLA at all. Moreover we are not fully sure how long halting takes, since ρ depends on the unknown $\mathbf{w}_f$, though in practice it is fast.

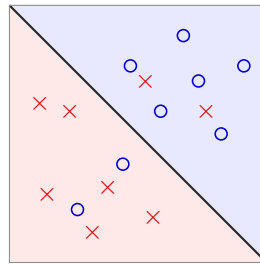
Question 2.4.1. What if $\mathcal{D}$ is not linearly separable?

In practice the usual reason is not that the truth is curved, but that the data is **noisy**: the recorded y_n is sometimes simply wrong, or two customers with identical records behaved differently. The learning flow gains one term.



Question 2.4.2. How can we at least get $g \approx f$ on a noisy $\mathcal{D}$?

§2.4.ii. A Line with Noise Tolerance



If the noise is ‘little’, then $y_n = f(\mathbf{x}_n)$ *usually*, and therefore $g \approx f$ on $\mathcal{D}$ means $y_n = g(\mathbf{x}_n)$ *usually*. That suggests giving up on perfection and asking for the fewest mistakes instead:

$$\mathbf{w}_g \leftarrow \arg \min_{\mathbf{w}} \sum_{n=1}^N \mathbb{I}[y_n \neq \text{sign}(\mathbf{w}^\top \mathbf{x}_n)].$$

Remark 2.4.3. This is exactly the right thing to ask for and, unfortunately, NP-hard to solve. The obstruction is the Iverson bracket: the objective counts, so it is piecewise constant, and there is no slope anywhere to follow downhill. Much of *How can machines learn?* is about replacing this count with a smoother surrogate; here we take the cheaper route instead and modify the algorithm rather than the objective.

§2.4.iii. The Pocket Algorithm

Question 2.4.4. Can we modify PLA to get an ‘approximately good’ g ?

The fix is almost embarrassingly simple. PLA already wanders among candidate lines; it just has no memory, and on non-separable data it wanders forever. So give it a pocket: keep the best line seen so far, and keep wandering.

Algorithm 2.3: Pocket Algorithm**Input:** training examples $\mathcal{D}$, an iteration budget**Output:** $\hat{\mathbf{w}}$, called $\mathbf{w}_{\text{POCKET}}$ and returned as g

```

1 initialize the pocket weights  $\hat{\mathbf{w}}$ ;
2 for  $t = 0, 1, \dots$  until enough iterations do
3   find a (random) mistake of  $\mathbf{w}_t$ , called  $(\mathbf{x}_{n(t)}, y_{n(t)})$ ;
4   (try to) correct the mistake by  $\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + y_{n(t)}\mathbf{x}_{n(t)}$ ;
5   if  $\mathbf{w}_{t+1}$  makes fewer mistakes on  $\mathcal{D}$  than  $\hat{\mathbf{w}}$  then  $\hat{\mathbf{w}} \leftarrow \mathbf{w}_{t+1}$ ;
6 return  $\hat{\mathbf{w}}$  as  $g$ ;

```

Note what changed and what did not. The update rule is untouched — the black lines of PLA are still there — and the only additions are the pocket and the iteration budget, which is now needed because there is no “no more mistakes” to stop on.

A simple modification of PLA to find (somewhat) ‘best’ weights.

Exercise 2.4.5 (Fun Time). Should we use pocket or PLA? Since we do not know whether $\mathcal{D}$ is linearly separable in advance, we may decide to just go with pocket instead of PLA. If $\mathcal{D}$ is actually linearly separable, what is the difference between the two?

- (1) pocket on $\mathcal{D}$ is slower than PLA;
- (2) pocket on $\mathcal{D}$ is faster than PLA;
- (3) pocket on $\mathcal{D}$ returns a better g in approximating f than PLA;
- (4) pocket on $\mathcal{D}$ returns a worse g in approximating f than PLA.

Reference Answer. (1). Because pocket needs to check whether $\mathbf{w}_{t+1}$ is better than $\hat{\mathbf{w}}$ in each iteration — a full pass over $\mathcal{D}$ — it is slower than PLA. On a linearly separable $\mathcal{D}$, $\mathbf{w}_{\text{POCKET}}$ ends up the same as $\mathbf{w}_{\text{PLA}}$, both making no mistakes; so the price of not knowing in advance is time, not quality. ■

Summary

Perceptron Hypothesis Set hyperplanes / linear classifiers in $\mathbb{R}^d$.

Perceptron Learning Algorithm (PLA) correct mistakes and improve iteratively.

Guarantee of PLA no mistake eventually, if linearly separable.

Non-Separable Data hold somewhat ‘best’ weights in a pocket.

We now own one complete learning model, $\mathcal{H}$ and $\mathcal{A}$ both. It answers a *yes/no* question, from a vector of *real* features, given *labelled* examples handed over *all at once*. Every one of those emphasised choices can be varied, and the next chapter takes stock of the resulting zoo of learning problems.

3. Types of Learning

Lecture 3

As previously seen . Section 2.2 produced one complete learning model: PLA takes a linearly separable $\mathcal{D}$ and the perceptrons $\mathcal{H}$, and returns a hypothesis g . That is one point in a very large space of possibilities.

16 Sep.

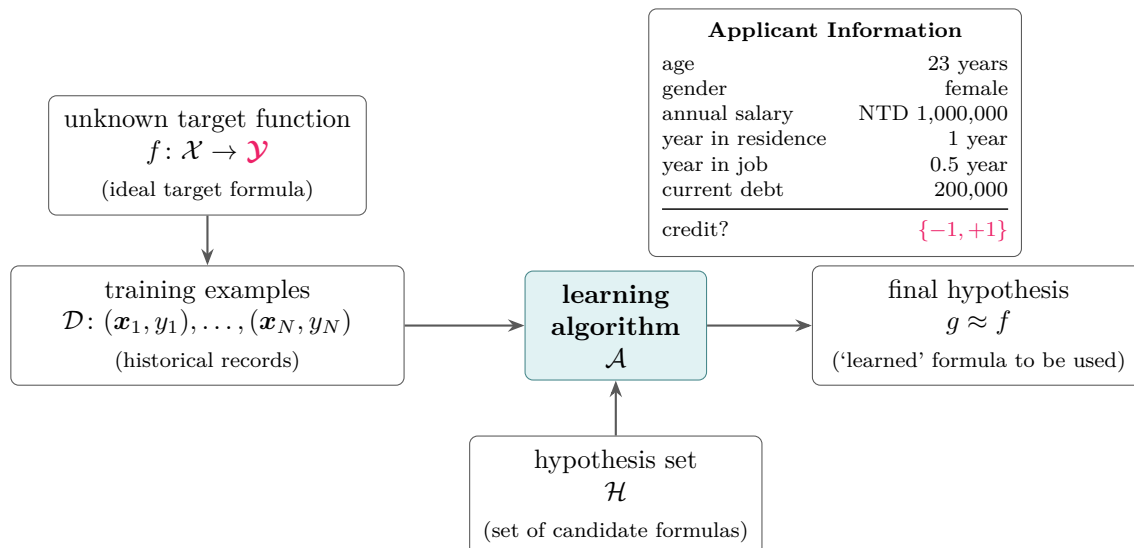
The frame of Section 1.4 has several slots — the output space $\mathcal{Y}$, the labels y_n , the way the data reaches us, the input space $\mathcal{X}$ — and so far every one of them has been filled in the simplest way available. This chapter varies them one at a time, which is the cheapest way to see what the frame can hold. Nothing here is proved; the point is a map of the territory, and the names on it recur for the rest of the course.

§3.1. Learning with Different Output Space $\mathcal{Y}$

— slides p.2–8 —

§3.1.i. The Credit Approval Problem Revisited

Credit approval answers one yes/no question, so its output space has exactly two elements.



$\mathcal{Y} = \{-1, +1\}$: **binary classification.**

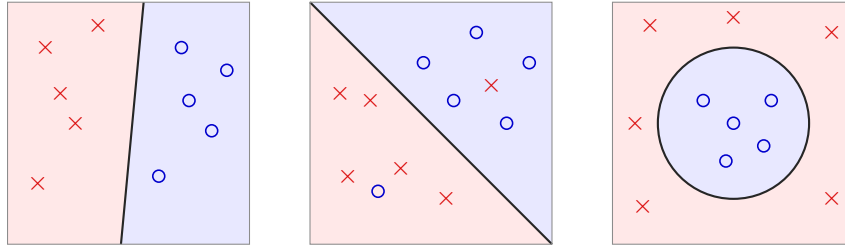
§3.1.ii. Binary Classification

Definition 3.1.1 (Binary classification). A learning problem whose output space has exactly two elements, $\mathcal{Y} = \{-1, +1\}$, is called **binary classification**. The two elements are only names: good/bad, spam/non-spam, sick/not sick.

The same output space covers an enormous range of questions:

- credit approve/disapprove;
- email spam/non-spam;
- patient sick/not sick;
- ad profitable/not profitable;
- answer correct/incorrect (KDDCup 2010).

Geometrically it is the picture we already know — a boundary with one class on each side, easy or hard depending on how the data falls.

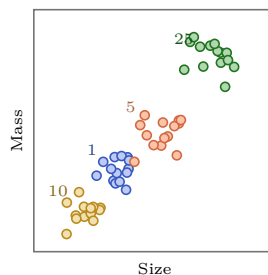


A core and important problem, with many tools that serve as building blocks of other tools.

§3.1.iii. Multiclass Classification

Definition 3.1.2 (Multiclass classification). A learning problem whose output space is finite and explicitly listed, $\mathcal{Y} = \{1, 2, \dots, K\}$ with $K \geq 2$, is called **multiclass classification**; Definition 3.1.1 is the special case $K = 2$.

Example 3.1.3 (Coin recognition). Classify US coins — 1c, 5c, 10c, 25c — by their (size, mass). The four denominations form four clusters, because a heavier coin is usually a larger one.



Here $\mathcal{Y} = \{1c, 5c, 10c, 25c\}$, so $K = 4$.

Other multiclass problems: written digits $\Rightarrow 0, 1, \dots, 9$; pictures $\Rightarrow$ apple, orange, strawberry; emails $\Rightarrow$ spam, primary, social, promotion, update (Google).

Many applications in practice, especially for ‘recognition’.

§3.1.iv. Regression

Definition 3.1.4 (Regression). A learning problem whose output space is the whole real line, $\mathcal{Y} = \mathbb{R}$, is called **regression**; if the output is confined to an interval, $\mathcal{Y} = [\text{lower}, \text{upper}] \subset \mathbb{R}$, it is **bounded regression**. The difference from classification is that $\mathcal{Y}$ now carries an order and a distance, so being wrong by a little is better than being wrong by a lot.

Keep the patient, change the question, and the output space changes with it:

- binary classification: patient features $\Rightarrow$ sick or not;
- multiclass classification: patient features $\Rightarrow$ which type of cancer;
- *regression*: patient features $\Rightarrow$ how many days before recovery.

Regression was studied deeply in statistics long before machine learning existed, which is why so many of its tools arrive ready-made. Other regression problems: company data $\Rightarrow$ stock price; climate data $\Rightarrow$ temperature.

Also core and important, with many ‘statistical’ tools as building blocks of other tools.

§3.1.v. Structured Learning

Definition 3.1.5 (Structured learning). **Structured learning** is a learning problem whose outputs are composite objects — a tag for every word of a sentence, a folding of a protein, a parse tree — whose parts constrain one another. Formally $\mathcal{Y}$ is still a finite set of ‘classes’, but an enormous one that is defined implicitly by those constraints rather than enumerated.

Example 3.1.6 (Sequence tagging). Multiclass classification sends a word to a word class. Structured learning sends a whole sentence to a structure — the class of every word at once:

$\underbrace{\text{I}}_{\text{pronoun}} \quad \underbrace{\text{love}}_{\text{verb}} \quad \underbrace{\text{ML}}_{\text{noun}}$

so that $\mathcal{Y} = \{\text{PVN}, \text{PVP}, \text{NVN}, \text{PV}, \dots\}$, not including VVVVV.

That exclusion is the whole difficulty. The output space is a huge multiclass problem — a structure is a kind of hyperclass — but without an *explicit* class definition: the valid structures are constrained by grammar, and nobody wrote the list down. Other structured problems: protein data $\Rightarrow$ protein folding; speech data $\Rightarrow$ speech parse tree.

A fancy but complicated learning problem.

§3.1.vi. Mini Summary

Note 3.1.7 (Learning with different output space $\mathcal{Y}$).

- binary classification: $\mathcal{Y} = \{-1, +1\}$;
- multiclass classification: $\mathcal{Y} = \{1, 2, \dots, K\}$;
- regression: $\mathcal{Y} = \mathbb{R}$;
- structured learning: $\mathcal{Y} = \text{structures}$;
- ... and a lot more!!

Core tools: binary classification and regression.

Exercise 3.1.8 (Fun Time). The entrance system of the school gym, which does automatic face recognition based on machine learning, is built to charge four different groups of users differently: Staff, Student, Professor, Other. What type of learning problem best fits the need of the system?

- (1) binary classification;
- (2) multiclass classification;
- (3) regression;
- (4) structured learning.

Reference Answer. (2). There is an ‘explicit’ $\mathcal{Y}$ that contains four classes. ■

§3.2. Learning with Different Data Label y_n

— slides p.9–15 —

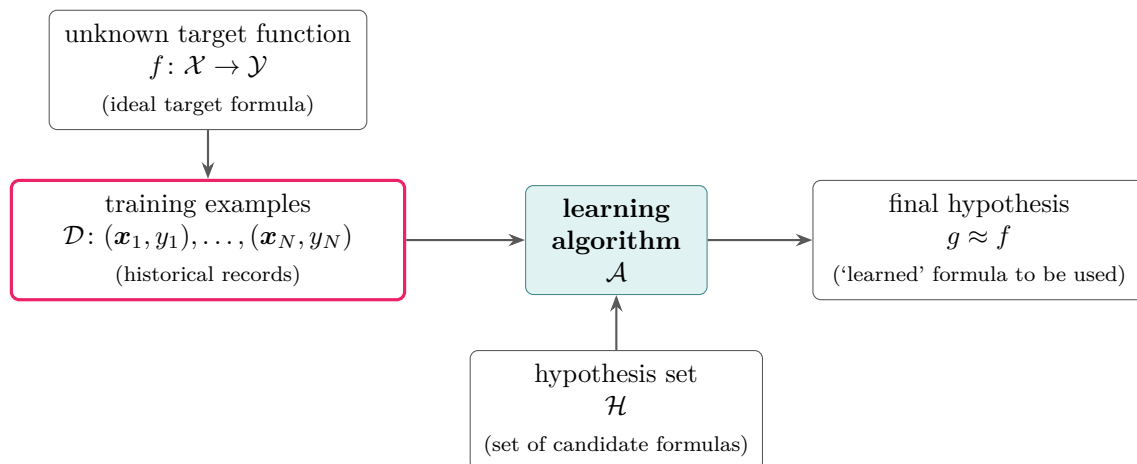
§3.2.i. Supervised Learning

Definition 3.2.1 (Supervised learning). Learning from data in which every input carries its own label,

$$\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N,$$

is called **supervised learning**. This is the only kind we have used so far.

Everything so far has quietly assumed that the data arrives fully labelled.



Supervised learning: every $\mathbf{x}_n$ comes with a corresponding y_n .

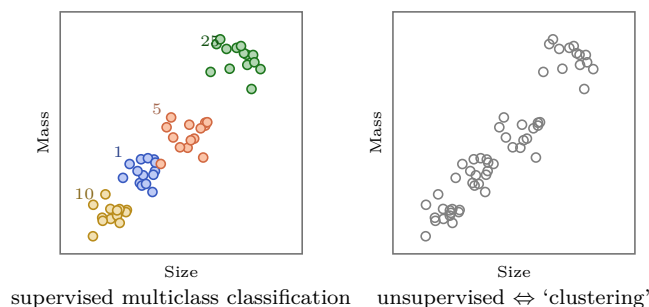
§3.2.ii. Unsupervised Learning

Definition 3.2.2 (Unsupervised learning). Learning from data that carries no labels at all,

$$\mathcal{D} = \{\mathbf{x}_n\}_{n=1}^N,$$

is called **unsupervised learning**. With no y_n there is no target f to approximate in the sense of Section 1.4, so what counts as a good g has to be decided separately for each problem.

Now delete the labels. The coins are still there, and they still fall into clusters — but nothing says which cluster is the quarter.



Other clustering problems: articles $\Rightarrow$ topics; consumer profiles $\Rightarrow$ consumer groups. Clustering is only one of several unsupervised problems, and they are best remembered as unsupervised shadows of the supervised ones — each keeps the *shape* of a supervised problem but has to invent its own notion of a right answer.

Definition 3.2.3 (Clustering). **Clustering** learns a map

$$\{\mathbf{x}_n\} \Rightarrow \text{cluster}(\mathbf{x})$$

assigning each input to one of finitely many groups, where the groups themselves are discovered rather than given. It is roughly ‘unsupervised multiclass classification’, with the essential difference that the classes have no names and no prescribed number — articles $\Rightarrow$ topics.

Definition 3.2.4 (Density estimation). **Density estimation** learns a map

$$\{\mathbf{x}_n\} \Rightarrow \text{density}(\mathbf{x})$$

reporting how tightly the inputs crowd around each point of $\mathcal{X}$. Since a density is a non-negative number, this is roughly ‘unsupervised bounded regression’ — traffic reports with location $\Rightarrow$ dangerous areas.

Definition 3.2.5 (Outlier detection). **Outlier detection** learns a map

$$\{\mathbf{x}_n\} \Rightarrow \text{unusual}(\mathbf{x})$$

flagging the inputs that do not look like the rest. It is roughly an extreme ‘unsupervised binary classification’: extreme because the two classes are wildly unbalanced — almost everything is normal, and it is the rare case that matters — so accuracy is a useless measure of success. Internet logs $\Rightarrow$ intrusion alert.

... and a lot more!!

Unsupervised learning: diverse, with possibly very different performance goals.

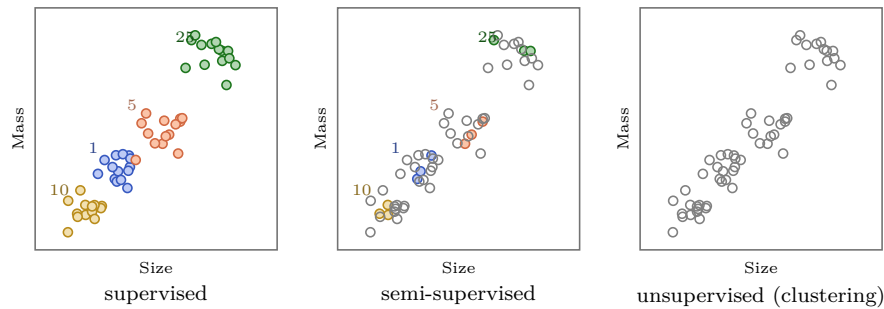
§3.2.iii. Semi-supervised Learning

Definition 3.2.6 (Semi-supervised learning). Learning from data in which only a few inputs carry labels,

$$\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^{\ell} \cup \{\mathbf{x}_n\}_{n=\ell+1}^N, \quad \ell \ll N,$$

is called **semi-supervised learning**. The unlabelled majority is not useless: it shows where the inputs lie, and a boundary is more believable if it passes through a thin region of them.

Between the two extremes sits the case that is commonest in practice: labels exist, but only for a few examples, because labelling costs money.



Other semi-supervised problems: face images with a few labelled $\Rightarrow$ face identifier (Facebook); medicine data with a few labelled $\Rightarrow$ medicine effect predictor.

Semi-supervised learning: leverage unlabelled data to avoid ‘expensive’ labelling.

§3.2.iv. Reinforcement Learning

Definition 3.2.7 (Reinforcement learning). Learning from data of the form

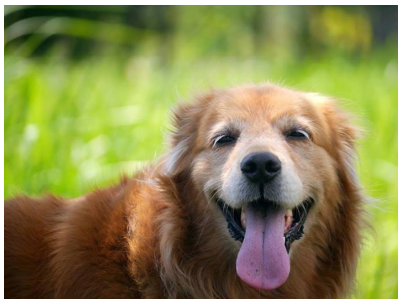
$$\mathcal{D} = \{(\mathbf{x}_n, \tilde{y}_n, \text{goodness}(\tilde{y}_n))\}_{n=1}^N,$$

where $\tilde{y}_n$ is an action that was actually taken and the third entry says only how well it turned out, is called **reinforcement learning**. The correct label y_n is never revealed — one learns that a choice was good or bad, not what the right choice would have been.

A ‘very different’ but natural way of learning, and the easiest way in is the one the slides take.

Example 3.2.8 (Teach your dog to sit). You say ‘sit down’.

- *The dog pees on the ground.* BAD DOG. THAT’S A VERY WRONG ACTION. You cannot easily show the dog that $y_n = \text{sit}$ when $\mathbf{x}_n = \text{‘sit down’}$, but you can *punish* it, which says that $\tilde{y}_n = \text{pee}$ is wrong.
- *The dog sits down.* GOOD DOG. LET ME GIVE YOU SOME COOKIES. You still cannot show that $y_n = \text{sit}$, but you can *reward* it, which says that $\tilde{y}_n = \text{sit}$ is good.



Note what the training signal is. It is not the correct label y_n but a triple $(\mathbf{x}, \tilde{y}, \text{goodness})$: some action was taken, and we learn only how good *that* action was — never what the best one would have been. Other problems of the same shape: (customer, ad choice, ad click earning) $\Rightarrow$ ad system; (cards, strategy, winning amount) $\Rightarrow$ black jack agent.

Reinforcement: learn with ‘partial/implicit information’ (often sequentially).

§3.2.v. Mini Summary

Note 3.2.9 (Learning with different data label y_n).

- supervised: all y_n ;
- unsupervised: no y_n ;
- semi-supervised: some y_n ;
- reinforcement: implicit y_n by goodness($\tilde{y}_n$);
- ... and more!!

Core tool: supervised learning.

Exercise 3.2.10 (Fun Time). To build a tree recognition system, a company decides to gather one million pictures on the Internet. Then it asks each of its 10 members to view 100 pictures and record whether each picture contains a tree. The pictures and records are then fed to a learning algorithm to build the system. What type of learning problem does the algorithm need to solve?

- (1) supervised;
- (2) unsupervised;
- (3) semi-supervised;
- (4) reinforcement.

Reference Answer. (3). The 1,000 records are the labelled $(\mathbf{x}_n, y_n)$; the other 999,000 pictures are unlabelled $\mathbf{x}_n$. ■

§3.3. Learning with Different Protocol $f \Rightarrow (\mathbf{x}_n, y_n)$

— slides p.16–21 —

The third slot is not about what the data *is* but about how it reaches the algorithm — the arrow from f down to $\mathcal{D}$.

§3.3.i. Batch Learning

Definition 3.3.1 (Batch learning). Under the **batch** protocol the entire $\mathcal{D}$ is handed to $\mathcal{A}$ at once, and $\mathcal{A}$ returns a single hypothesis g that is then held fixed.

The coin recogniser of [Example 3.1.3](#) is handed its whole data set at once and learns from all of it: batch supervised multiclass classification. Other examples: a batch of (email, spam?) $\Rightarrow$ spam filter; a batch of (patient, cancer) $\Rightarrow$ cancer classifier; a batch of patient data $\Rightarrow$ groups of patients — the last one unsupervised, so the protocol is independent of the previous two sections.

Batch learning: a very common protocol.

§3.3.ii. Online Learning

Definition 3.3.2 (Online learning). Under the **online** protocol the examples arrive one at a time: at step t the learner sees $\mathbf{x}_t$, predicts with its current g_t , is then told y_t , and updates to g_{t+1} . There is no separate training phase — the hypothesis in use is the one still being learned.

Example 3.3.3 (A spam filter that improves). A *batch* spam filter learns from known (email, spam?) pairs and then predicts with a fixed g . An *online* spam filter instead repeats, for $t = 1, 2, \dots$:

1. observe an email $\mathbf{x}_t$;
2. predict its spam status with the current $g_t(\mathbf{x}_t)$;
3. receive the ‘desired label’ y_t from the user, and then update g_t with $(\mathbf{x}_t, y_t)$.

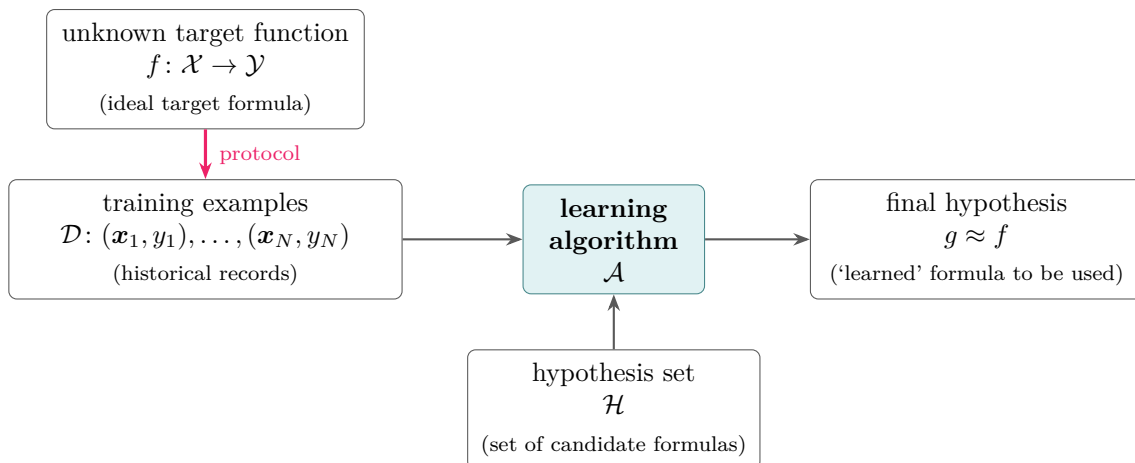
Question 3.3.4. Two connections to what we have already learned: PLA can be easily adapted to the online protocol — how? And reinforcement learning is often done online — why?

Answer. PLA already works one example at a time: it looks at a single $(\mathbf{x}_{n(t)}, y_{n(t)})$, and either leaves $\mathbf{w}$ alone or corrects it. Feed it a stream instead of a fixed $\mathcal{D}$ and nothing in [algorithm 2.1](#) has to change. Reinforcement learning is online almost by definition: the signal $(\mathbf{x}, \tilde{y}, \text{goodness})$ only exists once an action has been taken, so the data cannot be collected in advance — it is generated by the learner’s own behaviour. ■

Online: the hypothesis ‘improves’ through receiving data instances sequentially.

§3.3.iii. Active Learning

Definition 3.3.5 (Active learning). Under the **active** protocol the learner *chooses* which $\mathbf{x}_n$ it wants labelled and queries that label, instead of accepting whichever examples happen to arrive. The aim is to reach a good g from fewer labels by asking about the inputs that are most informative.



Note 3.3.6 (Protocol $\Leftrightarrow$ learning philosophy).

- batch: ‘duck feeding’ — everything is pushed in at once;
- online: ‘passive sequential’ — take what arrives, in order;

- active: ‘question asking’ (sequentially) — query the y_n of a *chosen* x_n .

Active: improve the hypothesis with fewer labels (hopefully) by asking questions strategically.

§3.3.iv. Mini Summary

Note 3.3.7 (Learning with different protocol $f \Rightarrow (x_n, y_n)$).

- batch: all known data;
- online: sequential (passive) data;
- active: strategically-observed data;
- ... and more!!

Core protocol: batch.

Exercise 3.3.8 (Fun Time). A photographer has 100,000 pictures, each containing one baseball player. He wants to categorize the pictures automatically by the player inside. He starts by categorizing 1,000 pictures himself, and then writes an algorithm that tries to categorize the other pictures if it is ‘confident’ on the category, while pausing for — and learning from — human input if not. What protocol best describes the nature of the algorithm?

- (1) batch;
- (2) online;
- (3) active;
- (4) random.

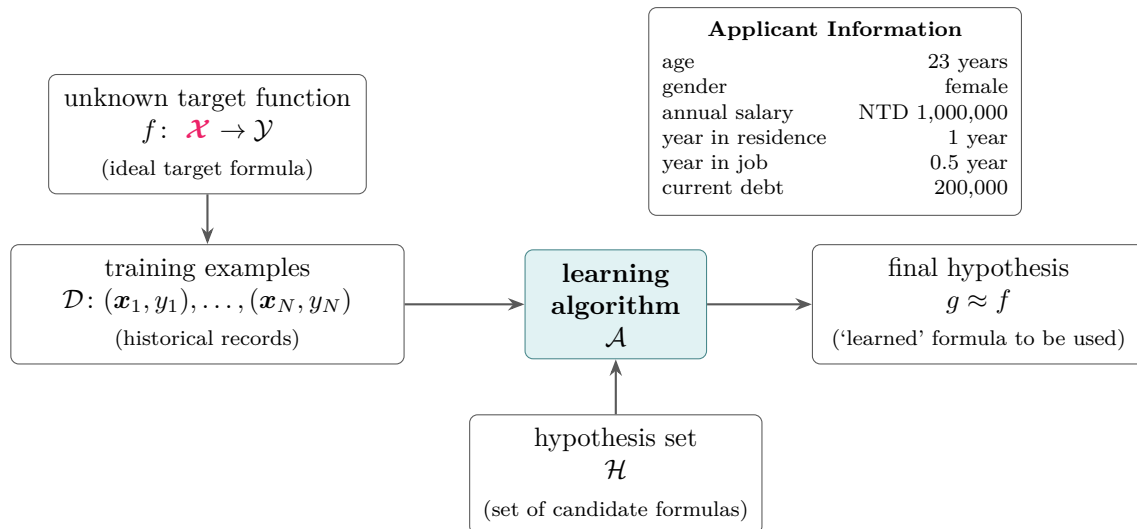
Reference Answer. (3). The algorithm takes an active but naïve strategy: ask when ‘confused’. You should probably do the same when taking a class. :-)

§3.4. Learning with Different Input Space $\mathcal{X}$

— slides p.22–28 —

§3.4.i. Concrete Features

Definition 3.4.1 (Concrete features). $\mathcal{X} \subseteq \mathbb{R}^d$ consists of **concrete features** when each of its d dimensions carries a sophisticated physical meaning — annual salary, mass, tumour size — normally because a human who understood the task chose to measure it.



Concrete features: each dimension of $\mathcal{X} \subseteq \mathbb{R}^d$ represents ‘sophisticated physical meaning’.

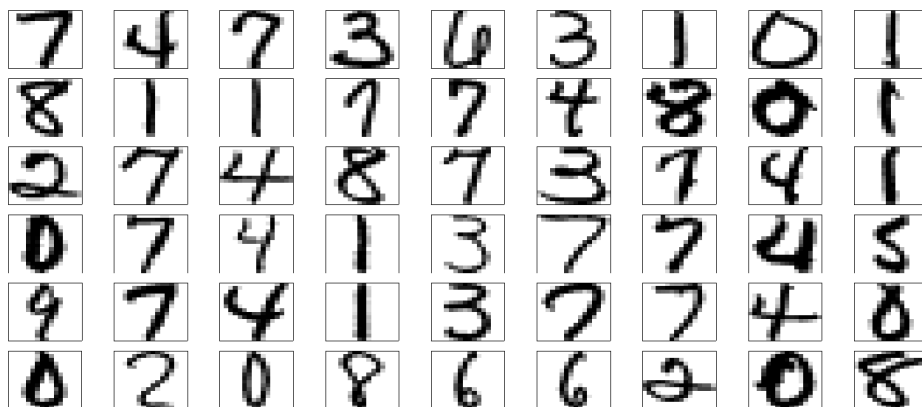
Every input we have met so far is of this kind: (size, mass) for coin classification, customer information for credit approval, patient information for cancer diagnosis. What they share is that somebody decided which quantities were worth measuring — the features carry ‘human intelligence’ about the learning task, put there before learning began.

Concrete features: the ‘easy’ ones for ML.

§3.4.ii. Raw Features

Definition 3.4.2 (Raw features). $\mathcal{X}$ consists of **raw features** when each dimension has only a simple physical meaning — the darkness of one pixel, one sample of a waveform. Any dimension on its own says almost nothing; the information lives in combinations of very many of them.

Example 3.4.3 (Digit recognition). Features $\Rightarrow$ meaning of digit: a typical supervised multi-class classification problem. But what are the features?

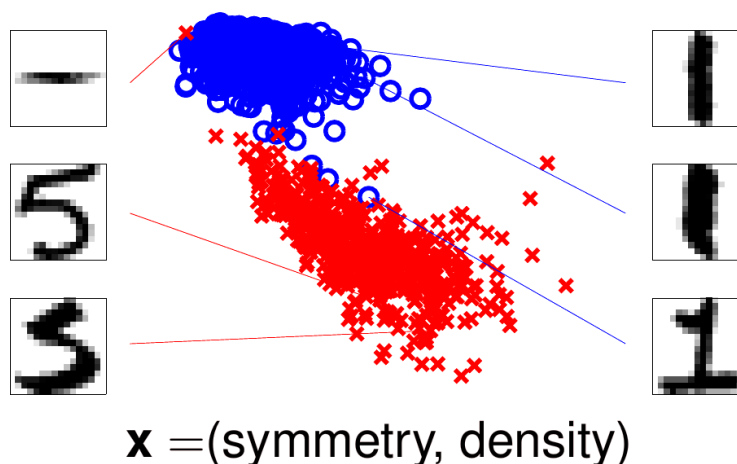


The honest answer is that a 16×16 grey image is just a list of 256 numbers,

$$\mathbf{x} \equiv (0, 0, 0.9, 0.6, \dots) \in \mathbb{R}^{256},$$

and each of those numbers means only ‘how dark is this one pixel’. That is a *simple* physical meaning, and it makes the problem harder for machine learning, not easier — no single pixel

tells you anything about the digit. Compare what a human would hand over instead: two numbers, symmetry and density, chosen because they happen to separate the classes.



Other problems with raw features: image pixels, speech signals, and so on.

Raw features: often need humans or machines to convert them to concrete ones.

§3.4.iii. Abstract Features

Definition 3.4.4 (Abstract features). $\mathcal{X}$ consists of **abstract features** when its dimensions have no (or very little) physical meaning at all — identifiers such as a userid or an itemid, where even the ordering of the numbers is an accident of bookkeeping.

Example 3.4.5 (Rating prediction, KDDCup 2011). Given previous (userid, itemid, rating) tuples, predict the rating that some userid would give to some itemid. This is a regression problem with $\mathcal{Y} \subseteq \mathbb{R}$ as the rating and $\mathcal{X} \subseteq \mathbb{N} \times \mathbb{N}$ as (userid, itemid).

User 5,001 is not in any sense one more than user 5,000, so this is harder still than raw features. Other problems with abstract features: student IDs in an online tutoring system (KDDCup 2010), advertisement IDs in an online ad system. That these two competitions were won by extracting factors from bare IDs is exactly the story of [Section 1.3](#).

Abstract: again needs ‘feature conversion/extraction/construction’.

§3.4.iv. Mini Summary

Note 3.4.6 (Learning with different input space $\mathcal{X}$).

- concrete: sophisticated (and related) physical meaning;
- raw: simple physical meaning;
- abstract: no (or little) physical meaning;
- ... and more!!

‘Easy’ input: concrete.

Exercise 3.4.7 (Fun Time). Consider the problem of building an online image advertisement system that shows users the most relevant images. What features can you choose to use?

- (1) concrete;
- (2) concrete, raw;
- (3) concrete, abstract;
- (4) concrete, raw, abstract.

Reference Answer. (4). Concrete user features, raw image features, and maybe abstract user/image IDs. ■

Summary

Output space $\mathcal{Y}$ *classification, regression, structured.*

Data label y_n *supervised, un/semi-supervised, reinforcement.*

Protocol $f \Rightarrow (x_n, y_n)$ *batch, online, active.*

Input space $\mathcal{X}$ *concrete, raw, abstract.*

The emphasised entry in each line is the one this course treats as its core tool, and the combination of all four — batch supervised binary classification on concrete features — is exactly the setting PLA already solves. Everything else in the list is reached by relaxing one slot at a time.

Which makes the next question awkward. We have a solved case and a map of its neighbours, but not one word yet about whether any of it works on data we have never seen. The next chapter argues, seriously, that learning is impossible.